

DEC 2026 Pre-Traffic Decision Engine Build Spec

Product objective

Turn DEC from a strong audit app into a reliable **pre-traffic launch decision engine** that tells a business owner, operator, or media buyer:

- whether an asset is safe to launch
- whether it is safe only to test cautiously
- whether it is unsafe to scale
- exactly what must be fixed before spend

The system should be optimized for a 2026 environment where AI-generated copy is abundant, buyer skepticism is elevated, and generic fluency is no longer persuasive enough on its own.

Diagnosis of the current system

What the current architecture is already doing well

The current `audit.js` and `index.html` already provide:

- strict JSON output discipline
- useful persuasion dimensions
- campaign-fit support
- signal pre-processing
- a serious operator-facing UI with launch-verdict framing

That means the base is viable.

The main architectural problem

The current engine relies too heavily on a **single-pass model judgment** to do all of the following at once:

- persuasion analysis
- proof evaluation
- claim exposure review
- launch readiness assessment
- campaign coherence analysis
- final verdict issuance

This creates elegant output, but it limits reliability, consistency, and precision.

Design conclusion

DEC should move from a **single audit prompt** to a **multi-engine tribunal architecture** with deterministic final

verdict resolution.

New product standard

DEC should stop trying to answer:

| Is this good copy?

And instead answer:

| Where will this fail when traffic hits it, and what must change before budget is committed?

That is the category-defining position.

Core system architecture

Layer 1 — Intake Normalization Engine

Purpose: understand what is being judged before scoring it.

Required normalized fields:

- asset_type
- business_type
- market_category
- channel
- traffic_source
- audience_temperature
- awareness_level
- offer_model
- price_point
- sales_cycle
- claim_sensitivity
- competitive_maturity
- funnel_stage
- brand_proof_available
- campaign_mode (single asset / campaign fit)

Why this matters:

The same asset should not be judged the same way for:

- cold paid traffic
- warm email
- sales page retargeting
- local lead gen
- info product webinar
- ecommerce PDP copy

Layer 2 — Persuasion Engine

Purpose: assess whether the copy can create attention, belief progression, desire, and action.

Recommended dimensions:

- hook
- lead
- body
- mechanism
- offer
- n-cta
- positioning
- objection_handling
- urgency_quality
- readability_flow
- belief_progression

Primary question answered:

| Does this persuade?

Layer 3 — Proof Strength Index (PSI)

Purpose: measure whether belief is earned strongly enough for launch.

Recommended proof dimensions:

- claim_specificity
- mechanism_substantiation
- product_validation
- operational_verifiability
- testimonial_quality
- authority_quality
- evidence_uniqueness
- traceability_auditability
- substantiation_visibility

Primary question answered:

| Is this proof strong enough for skeptical traffic?

Layer 4 — Skepticism & AI-Fatigue Engine

Purpose: simulate how a 2026 buyer or operator will resist the asset.

Recommended outputs:

- ai_pattern_risk
- commodity_positioning_risk
- agreement_without_action_risk
- skepticism_pressure_score
- reader_resistance_points
- genericity_flags
- false_distinctness_flags

Primary question answered:

| Will this be dismissed as familiar, synthetic, or unearned?

Layer 5 — Claim & Launch Exposure Engine

Purpose: assess risk exposure before spend.

Recommended outputs:

- overall_claim_risk
- performance_claims_present
- implied_superiority_claims_present
- safety_claims_present
- guarantee_language_present
- disclosure_visibility
- substantiation_gap_risk
- platform_risk_notes

Primary question answered:

| Is this safe enough to put in front of traffic?

Layer 6 — Decision Synthesis Engine

Purpose: convert subsystem outputs into a final decision.

Recommended outputs:

- launch_verdict
- verdict_confidence
- copy_strength_score
- launch_safety_score
- safe_to_test
- safe_to_scale
- primary_blocker
- highest_risk_failure_mode
- required_fixes_before_launch
- required_fixes_before_scale
- watch_signals

Primary question answered:

| Should this launch now, test carefully, or be held?

New scoring model

Separate two top-level scores

DEC should show two distinct outcomes:

- . **Copy Strength Score** — how persuasive the asset is
- !. **Launch Safety Score** — how safe it is to commit spend

These must not be blended invisibly.

Recommended weighted model

Copy Strength Score

- Hook / Lead / Body / Mechanism / Offer / CTA / Positioning / Flow

Launch Safety Score

Weighted components:

- Proof Strength Index: 25%
- Offer Strength: 15%
- Persuasion Quality: 20%
- Skepticism Resistance: 15%
- Distinctness / Anti-Commodity: 10%
- Claim / Exposure Risk: 15%

Gating rules

Final verdict must not be a simple average.

Use hard gates.

Examples:

- If PSI overall < 6 on cold traffic, no full GO verdict
 - If product_validation < 5 and the asset relies on superiority framing, no full GO verdict
 - If ai_pattern_risk = High in a mature category, reduce scale-readiness ceiling
 - If overall_claim_risk = Elevated and substantiation_visibility = Low, force Conditional Go or Hold
 - If offer score < 6, no scale-ready verdict even with strong copy
-

Verdict ladder

Use deterministic verdict classes:

- No-Go
- Hold
- Conditional Go
- Test-Only Go
- Go
- Scale-Ready

Suggested interpretation

- **No-Go**: major structural failure; unsafe to spend
- **Hold**: potentially viable but missing critical belief or risk controls
- **Conditional Go**: usable if conditions are fixed first or traffic is limited
- **Test-Only Go**: safe to run controlled testing, not safe to scale
- **Go**: commercially ready for launch
- **Scale-Ready**: evidence, offer, and persuasion are strong enough for confident spend expansion

New schema shape

```
{
  "mode": "single",
  "asset_type": "sales_page",
  "launch_verdict": "Conditional Go",
  "verdict_confidence": "High",
  "copy_strength_score": 84,
  "launch_safety_score": 68,
  "safe_to_test": true,
  "safe_to_scale": false,
  "primary_blocker": "Product-level proof insufficiency",
  "highest_risk_failure_mode": "Agreement without belief",
  "intake_context": {
    "channel": "paid_social",
    "audience_temperature": "cold",
    "awareness_level": "problem_aware",
    "claim_sensitivity": "moderate",
    "competitive_maturity": "high"
  },
  "persuasion_scores": {
    "hook": 9,
    "lead": 9,
    "body": 8,
    "mechanism": 8,
    "offer": 7,
    "cta": 8,
    "positioning": 7,
    "objection_handling": 6,
    "urgency_quality": 6,
  }
}
```

```
"belief_progression": 8,
"readability_flow": 9
},
"proof_strength_index": {
  "overall": 5,
  "claim_specificity": 6,
  "mechanism_substantiation": 6,
  "product_validation": 3,
  "operational_verifiability": 4,
  "testimonial_quality": 5,
  "authority_quality": 3,
  "evidence_uniqueness": 4,
  "traceability_auditability": 4,
  "substantiation_visibility": 4
},
"skepticism_engine": {
  "ai_pattern_risk": "Moderate",
  "commodity_positioning_risk": "Moderate",
  "agreement_without_action_risk": "High",
  "skepticism_pressure_score": 7,
  "reader_resistance_points": [
    "The mechanism sounds smart but not uniquely proven for this product.",
    "The trust promise feels stronger than the evidence supporting it."
  ],
  "genericity_flags": [
    "generalized trust language",
    "familiar category contrast"
  ],
  "false_distinctness_flags": []
},
"claim_exposure": {
  "overall_claim_risk": "Elevated",
  "performance_claims_present": true,
  "implied_superiority_claims_present": true,
  "safety_claims_present": false,
  "guarantee_language_present": true,
  "disclosure_visibility": "Not clear",
  "substantiation_gap_risk": "High",
  "platform_risk_notes": [
    "Trust and superiority language may require clearer substantiation in cold traffic environments."
  ]
},
"required_fixes_before_launch": [
  "Add direct product-level proof tied to the core mechanism.",
  "Increase substantiation visibility where superiority or trust claims are strongest.",
  "Strengthen offer differentiation for cold traffic."
```

```
],  
  "required_fixes_before_scale": [  
    "Reduce AI-pattern familiarity in the opening argument.",  
    "Add third-party or operational proof to harden scale-readiness."  
  ],  
  "watch_signals": [  
    "The page may generate agreement without enough belief to maximize  
conversion efficiency."  
  ]  
}
```

Backend refactor plan for `audit.js`

Keep

- environment config
- input guards
- model client initialization
- JSON schema discipline
- single asset vs campaign mode split
- text sanitization and normalization

Replace or expand

1. Replace one giant audit prompt with subsystem prompts

Create separate prompt modules:

- `PERSUASION_AUDIT_PROMPT`
- `PROOF_STRENGTH_PROMPT`
- `SKEPTICISM_ENGINE_PROMPT`
- `CLAIM_EXPOSURE_PROMPT`
- `DECISION_SYNTHESIS_PROMPT`

Each subsystem should produce structured JSON only.

2. Add a pre-model heuristic classifier

Expand `detectSignals()` to classify:

- claim verbs
- comparative statements
- proof markers
- numbers / quantification
- testimonial markers
- authority markers
- guarantee markers
- urgency markers
- compliance-sensitive phrases
- AI-pattern cadence markers
- distinctness markers
- offer-stack markers

These signals should be passed to the subsystem prompts as structured hints, not only raw text.

3. Introduce evidence classification

Before proof scoring, classify proof present in the asset:

- none
- anecdotal
- testimonial
- quantified
- expert
- process / operational
- product validation
- mechanism substantiation
- authority / third-party
- disclosure / substantiation support

This gives proof scoring far more precision.

4. Make the final verdict deterministic in code

Do not let one model decide the final launch verdict.

Use code rules such as:

- if `proof_strength_index.overall < 6` and `audience_temperature === "cold"` , ceiling verdict = `Conditional Go`
- if `offer < 6` , ceiling verdict = `Test-Only Go`
- if `overall_claim_risk === "High"` and `substantiation_visibility <= 4` , verdict cannot exceed `Hold`
- if `ai_pattern_risk === "High"` and `competitive_maturity === "high"` , reduce scale flag to false

5. Separate single-asset and campaign-fit logic more clearly

Campaign-fit should have its own subsystem outputs:

- promise continuity
- mechanism continuity
- proof progression
- objection progression
- CTA progression
- audience-stage integrity
- message-match stability
- funnel pressure coherence

Suggested backend module map

```
/src
  /audit
    intake.js
    heuristics.js
    evidence-classifier.js
    persuasion-audit.js
    proof-strength.js
    skepticism-engine.js
    claim-exposure.js
    decision-synthesis.js
    verdict-resolver.js
    campaign-fit.js
    schemas.js
    prompts.js
```

Function flow

```
normalizeInput(payload)
  -> detectSignals(text)
  -> classifyEvidence(text, signals)
  -> runPersuasionAudit(context, text, signals)
  -> runProofStrengthAudit(context, text, signals, evidence)
  -> runSkepticismAudit(context, text, signals)
  -> runClaimExposureAudit(context, text, signals, evidence)
  -> synthesizeDecision(allSubsystemOutputs)
  -> resolveVerdictDeterministically(synthesis, context)
  -> return report JSON
```

UI evolution plan for **index.html**

Keep

- premium operator aesthetic
- verdict masthead
- decision summary
- blockers / actions layout
- campaign fit mode
- Copy Chief layer framing

Add

1. Dual-score block

Show both:

- Copy Strength Score
- Launch Safety Score

This increases trust immediately because users can see the difference between persuasive quality and spend-readiness.

2. Decision basis panel

Add a panel showing the five pillars that drove the verdict:

- Persuasion
- Proof
- Skepticism Resistance
- Offer Strength
- Claim Exposure

3. Buyer skepticism simulation panel

Examples of output:

- “This sounds polished, but I still do not know what makes it objectively better.”
- “The mechanism is interesting, but the proof does not lock it to the product.”
- “This reads smoothly, but parts of it feel familiar rather than category-defining.”

This becomes one of the most valuable panels in the product.

4. AI-pattern risk chip

Status display:

- Low
- Moderate
- High

This should not shame the user. It should simply explain whether the copy may be perceived as synthetic, generic, or too familiar in-market.

5. Tiered action stack

Split action items into:

- Must fix before launch
- Should fix before scale
- Monitor after launch

This makes the report more useful than a generic “improvement list.”

6. Failure mode panel

A single line that names what breaks first under pressure, such as:

- Agreement without belief
 - Belief without urgency
 - Interest without trust
 - Strong promise, weak proof
 - Good copy, unsafe claim exposure
-

New product behaviors that matter in 2026+

1. Anti-genericity enforcement

The tool should detect and penalize:

- broad category insights presented as unique mechanisms
- specificity that sounds precise but is not verifiable
- familiar AI emotional pacing
- “smart framing” without hard brand ownership

2. Distinctness pressure by market maturity

The same asset should be judged harder in mature markets.

Examples:

- supplements: harder
- skincare: harder
- SaaS: harder
- local service: easier
- niche B2B solution with low copy sophistication: different threshold

3. Scale-readiness should require proof density

A page can be good enough to test without being safe enough to scale.

Scale-readiness should require:

- stronger PSI
 - lower claim exposure
 - lower AI-pattern risk
 - clearer differentiation
-

Campaign-fit engine expansion

Campaign-fit should not merely summarize a funnel.

It should diagnose continuity pressure.

Recommended campaign dimensions:

- promise continuity
- mechanism carry-through
- proof progression
- objection progression
- audience-stage matching
- offer sequencing
- CTA consistency
- narrative pressure progression
- landing consistency

Sample output:

- `promise_drift` : Low / Moderate / High
 - `proof_reset_risk` : High
 - `cta_mismatch_risk` : Low
 - `stage_role_integrity` : Moderate
 - `campaign_primary_breakpoint` : “Ad makes a sharper promise than landing page proves”
-

Recommended implementation phases

Phase 1 — Refactor backend into subsystem architecture

Goal: keep the current UI, improve audit reliability first.

Deliverables:

- separate prompts
- subsystem schemas
- heuristic signal expansion
- evidence classifier

Phase 2 — Add deterministic verdict resolver

Goal: improve consistency and operator trust.

Deliverables:

- hard gates
- verdict ladder
- testable rules

Phase 3 — Add PSI and Skepticism Engine to the report

Goal: make the product meaningfully more valuable than generic copy scorers.

Deliverables:

- proof strength panel
- skepticism simulation panel
- AI-pattern risk output

Phase 4 — Upgrade UI to dual-score + decision-basis model

Goal: align UX to the new engine.

Deliverables:

- Copy Strength score card
- Launch Safety score card
- must-fix / should-fix panels
- failure mode display

Phase 5 — Expand campaign-fit into continuity diagnostics

Goal: make DEC useful across full funnel launch decisions.

Deliverables:

- campaign continuity engine
- breakpoint detection
- per-stage role analysis

Immediate practical build decisions

Decision 1

Do not rebuild the UI from scratch.

The current visual framing is already credible.

Decision 2

Do not try to fix precision by making one prompt longer.

Precision will come from decomposition, not prompt size.

Decision 3

Keep model outputs structured and narrow.

Let code own final verdict logic.

Decision 4

Treat proof as its own economy.

Do not leave it as one generic persuasion dimension.

Decision 5

Add a skepticism engine before adding more scoring dimensions.

That will create the biggest product-value jump in a 2026 environment.

Final product positioning

The mature DEC should be positioned as:

A pre-traffic launch decision system that identifies exactly where copy will fail under skepticism, proof pressure, and market saturation before spend is committed.

That is stronger than calling it a copy audit app.

Recommended next build output

The next working artifact should be a **technical implementation spec** containing:

- exact subsystem prompt outlines
- exact JSON schemas per subsystem
- verdict resolver rules in pseudocode
- field additions to the current intake form
- UI panel mapping to current HTML sections
- migration notes from the current single-pass architecture